HYDERABAD TESTING BOOTCAMP

Manual Testing on Lenskart.com & Automation Testing on EaseMyTrip.com

Nirmalya Kumar Mohanta

Graduate Engineer Trainee | Coforge | Emp ID: 85009868

Agenda

What this presentation covers

01

Testing Fundamentals

What, Why & When of software testing

02

Manual Testing — Lenskart.com

Test Plan, Scenarios, Test Cases, Defect Report & RTM

03

Automation Testing — EaseMyTrip.com

Tools, Framework Setup, Scripts & Execution Reports

What, Why & When of Testing

Foundations before diving into Lenskart & EaseMyTrip

W

WHAT?

Testing is the process of executing a program with the intent of finding defects, to ensure the product meets the client's quality requirements.

W

WHY?

Testing identifies defects early, ensures quality and reliability, and reduces the risk of failures that impact users and the business.

W

WHEN?

Testing runs throughout the SDLC — starting at requirement analysis and continuing through development, deployment, and maintenance.

SECTION 02

Manual Testing

Case Study: Lenskart.com

Manual Testing

Definition & Approach

- Manual testing is the process of testing software without using automation tools.
- Testers execute test cases manually to identify defects and verify behavior.
- Needed to understand the application from a user's perspective — to catch usability and visual issues automation cannot detect.
- Applied here on Lenskart.com to validate registration, login, browsing, profile, address, wishlist, search, product details and logout flows.

9

Test Cases Executed

100%

Pass Rate

3 Defects Logged

Test Plan

Lenskart.com — End to End Testing

Project Name	Lenskart.com
Module	End to End Test
Test Objective	To validate end-to-end functionality of Lenskart.com
Test Setup	MS Excel Sheet (Online)
Scope (In)	Registration, Login, Homepage, Profile, Address, Wishlist, Search, Product Details, Logout
Scope (Out)	Payment gateway, third-party integrations, performance/load testing
Reviewed By	Raghavendra BN
Executed By	Nirmalya Kumar Mohanta

Test Plan

Lenskart.com — End to End Testing

Test Plan for Lenskart.com Website

1. Test Plan Identifier

File Name: 1.0_Lenskart_v1.0.doc

- Document Name: Test Plan – Job Portal Application
- Version: LK_JP_1.0
- Date: 27-May-2026
- **Location:** Coforge Ltd, Hyderabad

2. Document History

REV	Author	Edits	Date
1.0	Nirmalya Kumar Mohanta, Graduate Engineer Trainee	Initial Draft	27/05/2026

3. Introduction

Lenskart.com is an online job portal application designed to help users search and apply for jobs based on skills, location, experience, and keywords. The application allows users to register, log in, create and update profiles, search for jobs, view job details, and apply for suitable opportunities. This test plan defines the testing approach and scope required to validate the functional behavior, usability, and reliability of the Lenskart.com application.

4. Test Items

Specifies **what components/modules** will be tested.
Example:

- User Registration Module
- User Login Module
- Dashboard Module
- Profile Management Module
- Education Details Module
- Skills Management Module
- Job Search Module
- Job Details Page
- Logout Functionality

5. **Features to be Tested**

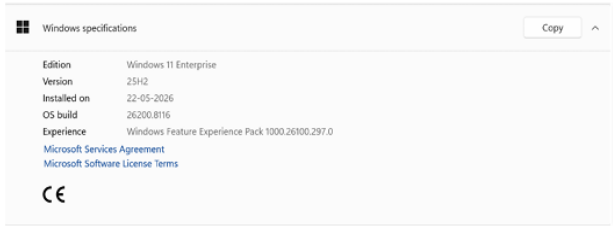
Medium priority features are features that have been stable and High Priority features are features that have been recently changed.

Feature to be Tested	Priority	Comments
User Registration	High	
User Login	High	
Dashboard Access	High	
Product Search	High	
Filters (Price, Frame, Color, etc.)	High	
Product Details View	High	
Add to Cart	High	
Cart Update (Qty, Remove)	High	
Checkout Process	High	
Payment Gateway	High	Integration Testing
Virtual Try-On	Medium	UI/Feature Heavy
Wishlist	Medium	
Order Tracking	High	
Profile Update	Medium	
Logout	High	

6. Features NOT to be Tested

- Help Contents and user manuals will not be tested.
- Installation and presentation will not be tested.
- Support of the other architectures also will not be tested.
- Third-party payment gateway internal logic
- Legacy modules not impacted by current release
- Experimental features

7. Test Environment



8. Test Approach / Test Strategy

Describes **how testing will be performed**.
The test strategy for Lenskart1.0 will include manual testing with automated testing **(Selenium With Java)** where time/need permits. Automated testing will be used to test the basic Interface objects on keyboard and mouse navigation, or any redundant tasks best translated to automation. Automated testing will be limited to a Win-64 platform because of Automation tool limitations.

- Sanity tests will include a very short battery of tests to ensure the Naukri1.0 runs/launches and can connect to the back-end database **(MySQL /SQL Server 2025).** Test execution will happen as individual component testing. These tests will be fully automated.
- **Acceptance tests** will include a short battery of tests that will only touch major features to ensure Naukri1.0 will operate smoothly at a basic level. These tests will be executed on **Windows 11 Enterprise**. The goal will be to automate all these tests.
- Full Functional tests will include a **comprehensive test** of all features listed in this document. These tests will be limited to Windows 11 Enterprise. These tests will include automation with time allowing.

8.1 Types of Testing

- Smoke Testing
- Sanity Testing
- Functional Testing
- Integration Testing
- System Testing
- Regression Testing

8.2 Test Levels

- Unit Testing (by developers)
- Integration Testing
- System Testing
- Acceptance Testing

8.3 Test Techniques

- Black Box Testing
- Boundary Value Analysis
- Equivalence Partitioning
- Error Guessing **Functionality Testing**

Test Plan — Scope & Approach (Lenskart.com)

Lenskart.com — End to End Testing

1. Objectives & Scope — Validate core Lenskart.com features: registration, login, homepage, profile, address, wishlist, search, product details, logout. Out of scope: payments, third-party integrations, performance testing.
2. Environment, Tools & Roles — Tested on Chrome browser; artifacts in MS Excel.
3. Entry/Exit Criteria & Risk — Entry: test cases reviewed. Exit: all 9 test cases executed, defects logged, RTM complete. Risk: live site UI changes may affect test steps.
4. Controlling Document — Governs Test Scenarios, Test Cases, Defect Report, and RTM for this case study.

The usability testing will be accomplished by verifying the information in each window are accurate. Menus, icons and toolbar functionality will be tested as applicable to the navigation and results panes. Multi Window Overlapping will be tested because product supports opening of multiple documents.

Robustness/Reliability Testing

Test the GUI for correct keyboard and mouse navigation, windows, menus, buttons, etc. These tests will include Keyboard navigation of Lenskart1.0, Mouse only navigation. Correctness and existence of warning/error dialog boxes. The correct/expected functionality of buttons, icons and menus.

Stress Testing

Not applicable

9. Entry and Exit Criteria

9.1 Entry Criteria

Conditions to start testing:

- Requirements approved
- Test environment ready
- Test data available
- Build deployed

9.2 Exit Criteria

Conditions to stop testing:

- All planned test cases executed
- Critical and high defects closed
- Test summary report prepared

10. Test Deliverables

Test plan	**1.0_ Lenskart.doc**
Test Specification	**1.0_ Lenskart.doc**
Test Automation Plan	TBD()
Test Checklist	TBD
Test Logs	TBD
Test tools	Selenium 4.38, Postman, Rest Assured, JMeter, Jenkins, Git and Git Hub and Appium and MySQL Server 2025

11. Testing Tasks & Resource Needs

- Use of MySQL Server database.
- Write Test Specification.
- Write Test Automation plan.
- Write Test Checklist.
- Setup workspace.
- Develop Automation script.

➢ Minimum requirements for platforms:

12. Schedule

Test Documentation

Document	*Development Time*		*Sign-off Date*	*Dependencies*
	Person-Days	*Completion Date*		
Test Plan	1day	27/05/2026	27/05/2026	Functional Spec. Review
Test Specifications	1 day	27/05/2026	27/05/2026	Test plan, Functional Spec, Review
Test Logs	TBD	27/05/2026		
Test Summary Report	TBD	27/05/2026		
Checklist	TBD	27/05/2026		

Test Development

Deliverable	*Development Time*		*Dependencies*
	Person-Days	*Completion Date*	
Functional Test Suite	1	27/05/2026	
Automation build			

Test Execution

Test Activity		*Execution Time (Person-Days)*			*Dependencies*
		Alpha	Beta	FCS	
Acceptance		1 Day	1 Day	1Days	
Functional	Functionality	1	1		
	Usability	All High to Medium test cases			
	Robustness				
	Stress				
	Multi Inst.				
Regression					
Documentation Review		1	1	1	
Total Test Cycle		1	1	1	

Test Scenarios

9 scenarios covering core Lenskart.com user journeys

Name: Nirmalya Kumar Mohanta Email ID: nirmalya.kool@gmail.com Emp ID: 85009868				
TEST SCENARIOS FOR LENSKART.COM				
Scenario ID	**Scenario Name**	**Scenario Description**	**Preconditions**	**Expected Outcome**
TS_01	Verify User Registration	Validate that a new user can register on Lenskart using valid details and OTP verification	User is not registered	User account is created successfully and user is logged in after OTP verification
TS_02	Verify User Login	Validate that a registered user can login using valid email/mobile and password/OTP	User is already registered	User is successfully logged in and redirected to homepage
TS_03	Homepage / Dashboard Module	Verify that homepage loads successfully with product listings, categories, banners, and search functionality	User is logged in	Homepage is displayed with products, offers, and navigation menu
TS_04	Profile Management Module	Validate that the user can view and update profile details like name, email, and mobile number	User is logged in	Profile details are updated successfully
TS_05	Address Management Module	Validate that user can add, edit, and save delivery address details	User is logged in	Address is saved successfully and available for checkout
TS_06	Wishlist Functionality	Verify that user can add and remove products from wishlist	User is logged in	Products are successfully added to and removed from wishlist
TS_07	Product Search Module	Validate that user can search products using keywords and apply filters like price, frame type, and color	User is on homepage	Relevant products are displayed based on search and filters
TS_08	Product Details Page	Verify that user can view detailed product information including images, price, description, and try-on feature	User selects a product	Product detail page is displayed with complete product information
TS_09	Logout Functionality	Validate that the user can logout from the application securely	User is logged in	User is logged out successfully and redirected to homepage

Test Cases

Sample executed test cases — Lenskart.com

Req_Id	Test_Case ID	Test_Case_Description	Test_steps	Test_Data	Expected_Result	Actual_Result	Status
Req_1.0	TC_01	Verify User Registration	1. User should navigate to https://www.lenskart.com/ 2. User should click on Sign Up / Login option on the Lenskart homepage 3. User should click on Create Account / Sign Up 4. User should enter valid Full Name, Email ID, Password, and Mobile Number 5. User should enter OTP received on mobile 6. User should click on Submit / Register button	Url: https://www.lenskart.com/ Name: Nirmalya Email: nirmalya.kool@gmail.com Password: ****** Mobile: +916371996914 City: Hyderabad	User account should be created successfully and user should be verified via OTP and redirected to homepage	User account created successfully and redirected to homepage	Passed
Req_1.1	TC_02	Verify User Login with Valid Credentials	1. User should open Microsoft Edge browser 2. User should navigate to https://www.lenskart.com/ 3. User should click on Login / Sign In option on the homepage 4. User should enter valid Email ID / Mobile number and Password / OTP 5. User should click on Login button	Url: https://www.lenskart.com/ Email: nirmalya.kool@gmail.com Password: ******	User should be logged in successfully and homepage/dashboard should be displayed	User logged in successfully and homepage displayed	Passed
Req_1.2	TC_03	Dashboard / Homepage	1. User should open Microsoft Edge browser 2. User should navigate to https://www.lenskart.com/ 3. User should login with valid credentials 4. User should observe homepage/dashboard	Url: https://www.lenskart.com/ Email: nirmalya.kool@gmail.com Password: ******	Homepage should be displayed with product categories, banners, search bar, and offers	Homepage displayed successfully with products and navigation options	Passed
Req_1.3	TC_04	Profile Management (My Account)	1. User should login to Lenskart 2. User should navigate to My Account / Profile section 3. User should edit profile details (name, mobile, email) 4. User should click on Save	Name: Nirmalya Mobile: +91 9876543210 Email: nirmalya.kool@gmail.com	Profile details should be updated successfully	Profile updated successfully	Passed

Test Cases

Sample executed test cases — Lenskart.com

Req_1.4	TC_05	Address Details Module	1. User should login to Lenskart 2. User should navigate to My Account → Address Book 3. User should click on Add New Address 4. User should enter address details 5. User should click on Save	Name: Nirmalya Address: Gachibowli, Hyderabad Pincode: 500032 Mobile: +91 9876543210	Address should be added successfully and available during checkout	Address added successfully	Passed
Req_1.5	TC_06	Wishlist Functionality	1. User should login to Lenskart 2. User should search for any product 3. User should click on Add to Wishlist (heart icon) 4. User should navigate to Wishlist	Product: Eyeglasses / Sunglasses	Selected product should be added to wishlist successfully	Product added to wishlist	Passed
Req_1.6	TC_07	Product Search Module	1. User should open Lenskart homepage 2. User should enter product keyword in search bar 3. User should apply filters (price, frame type, color) 4. User should click on search	Keyword: Eyeglasses Filter: Price < 2000	Relevant products should be displayed based on search and filters	Relevant products displayed successfully	Passed
Req_1.7	TC_08	Product Details Page	1. User should search for a product 2. User should click on any product 3. User should view product details	Product: Eyeglasses	Product detail page should display product image, description, price, and options like Try-On/Add to Cart	Product details displayed correctly	Passed
Req_1.8	TC_09	Logout Functionality	1. User should login to Lenskart 2. User should click on Profile icon 3. User should click on Logout	Email: nirmalya.kool@gmail.com	User should be logged out successfully and redirected to homepage	User logged out successfully	Passed

Defect Case Study Report

Issues identified during checkout/payment testing on Lenskart.com

ID	Defect Description	Priority	Severity
1	Invalid card number accepted while other payment fields (e.g., expiry date) still validate as correct — inconsistent validation behavior.	High	Critical
2	CVV field accepts 4-digit input even though only 3 digits should be allowed per validation rule.	High	Major
3	Selected payment method shows an excessively long transaction timeout (~12 hrs), unsuitable for the checkout flow.	Medium	Major

Status: New | Created By: Nirmalya Kumar Mohanta | Assigned To: Raghavendra BN

Requirement Traceability Matrix (RTM)

Mapping requirements to executed test cases — Lenskart.com

Req No.	TC ID	Requirement Description	Status
R_1.0	TC_01	New users can register with valid details & proper validation	Passed
R_1.1	TC_02	Registered users can login securely via email/mobile + password/OTP	Passed
R_1.2	TC_03	Homepage loads with categories, banners, search & navigation	Passed
R_1.3	TC_04	Users can view & update profile details	Passed
R_1.4	TC_05	Users can add, edit & manage delivery addresses	Passed
R_1.5	TC_06	Users can add/remove products from wishlist	Passed
R_1.6	TC_07	Users can search & filter products by price, type, color	Passed
R_1.7	TC_08	Product details show images, price, description, try-on	Passed
R_1.8	TC_09	Users can logout securely and return to homepage	Passed

Peer Review

Mapping requirements to executed test cases — Lenskart.com

Nirmalya Kumar Mohanta <nirmalya.kool@gmail.com> Sat, 20 Jun 2026 at 7:38 pm
To: Raghavendra Bn <raghu.astepahead@gmail.com>

Hi Raghavendra,

Hope you are doing well.

I have shared the test deliverables for peer review, including the Test Plan, Test Scenarios, Test Cases, Defect Report, and RTM.

Kindly review them and share your feedback or suggestions at your convenience.

Thank you for your time and support.

Regards,
Nirmalya Kumar Mohanta.
Coforge,
Trainee/85009868.

2 attachments

Testplan_Lenskart_NirmalyaKumarMohanta(85009868) (1).pdf
310 KB

Lenskart_Testing_NirmalyaKumarMohanta(85009868) (1).xlsx
1.8 MB

Raghavendra Bn <raghu.astepahead@gmail.com> Mon, 22 Jun 2026 at 9:19 am
To: Nirmalya Kumar Mohanta <nirmalya.kool@gmail.com>

Hi Nirmalaya,

Thank you very much for sending across the test deliverables. I have successfully received all the documents, including the Test Plan, Test Scenarios, Test Cases, Defect Report, and the Requirements Traceability Matrix (RTM). I appreciate the effort and attention to detail that has gone into preparing these materials.

I will thoroughly reviewed each of the documents to ensure that all aspects of the testing process have been adequately covered. My review will focus on the completeness and accuracy of the test cases, the clarity of the test scenarios, and the traceability between requirements and test cases as documented in the RTM. Additionally, I will examine the defect report to understand the issues identified so far and how they have been addressed.

If there are any particular sections or areas within these documents that you would like me to pay special attention to, please let me know. Your input will help me provide more targeted and constructive feedback.

I have completed my review, I will compile my observations and share detailed feedback with you. If I have any questions or require further clarification during the review process, I will reach out accordingly.

Thank you again for your hard work and for keeping me in the loop. Looking forward to collaborating further to ensure the quality and success of this project.

Good To Go

Best regards,
Raghavendra BN

SECTION 03

Automation Testing

Case Study: EaseMyTrip.com

Automation Testing

Definition & Approach

- Automation testing runs test cases automatically using tools and scripts to verify the application works correctly.
- It increases speed, accuracy, and reliability — especially valuable for repetitive and regression testing.
- Applied here on EaseMyTrip.com to automate key flows such as search, login, and booking validation using Selenium + TestNG.

Target Application

EaseMyTrip.com

Framework

Selenium + TestNG

Reporting

Allure Reports

Tools & Setup

Technology stack used for EaseMyTrip.com automation

JDK

JDK (Java)

21 (Latest Stable)

SEL

Selenium

4.39.0

TNG

TestNG

7.11

POI

Apache POI

5.5.1

PMN

Postman

11.70.0

ECL

Eclipse IDE

2022

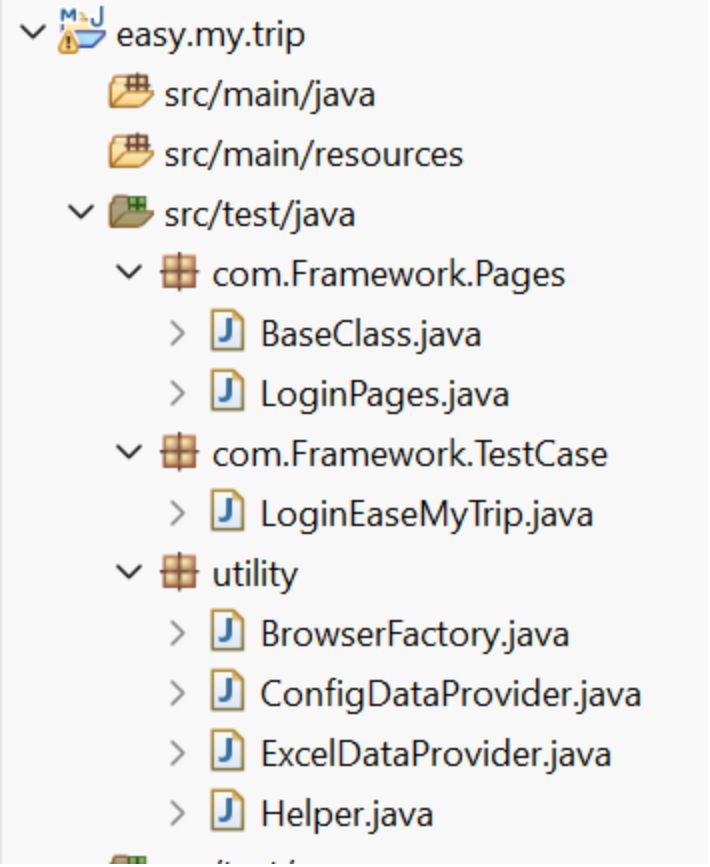
Framework Structure

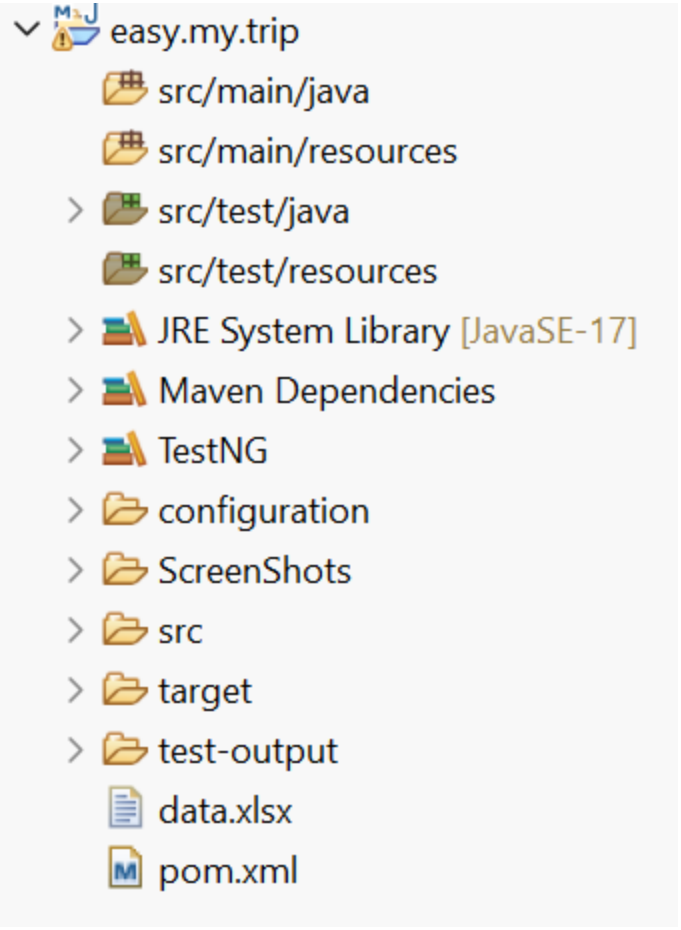
Project layout for EaseMyTrip.com automation suite

`pom.xml`	Maven dependencies — Selenium, TestNG, Apache POI, Allure
`src/main/java — BaseClass`	WebDriver setup, browser initialization & teardown
`src/main/java — BrowserFactory`	Manages browser instance creation (Chrome/Edge/Firefox)
`src/main/java — ConfigDataProvider`	Reads environment & configuration properties
`src/main/java — ExcelDataProvider`	Reads test data from Excel via Apache POI
`src/main/java — Page Objects`	EaseMyTrip page classes (Login, Search, Booking pages)
`src/test/java — Test Scripts`	TestNG test classes for each EaseMyTrip module
`test-output / allure-results`	Execution logs & Allure report data

Framework Structure

Project layout for EaseMyTrip.com automation suite





com.Framework.Pages/ com.Frameworkl.TestCase /com.utility — package layout

Framework – Maven Configuration (pom.xml)

Project layout for EaseMyTrip.com automation suite

```xml
<properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>

    <!-- Selenium -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.21.0</version>
    </dependency>

    <!-- TestNG -->
    <dependency>
        <groupId>org.testng</groupId>
        <artifactId>testng</artifactId>
        <version>7.11.0</version>
    </dependency>

    <!-- Apache POI -->
    <dependency>
        <groupId>org.apache.poi</groupId>
        <artifactId>poi</artifactId>
        <version>5.2.5</version>
    </dependency>

    <dependency>
        <groupId>org.apache.poi</groupId>
        <artifactId>poi-ooxml</artifactId>
        <version>5.2.5</version>
    </dependency>
```

Project Object Model – Base Class & Home Page

Project layout for EaseMyTrip.com automation suite

```java
package com.Framework.Pages;

import org.openqa.selenium.WebDriver;
import org.testng.ITestResult;
import org.testng.annotations.AfterClass;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeSuite;

import utility.BrowserFactory;
import utility.ConfigDataProvider;
import utility.ExcelDataProvider;
import utility.Helper;

public class BaseClass
{
    public WebDriver driver;
    public ExcelDataProvider excel;
    public ConfigDataProvider config;
```

```java
package com.Framework.Pages;

import java.time.Duration;

public class LoginPages {

    WebDriver driver;
    WebDriverWait wait;

    public LoginPages(WebDriver driver) {
        this.driver = driver;
        wait = new WebDriverWait(driver, Duration.ofSeconds(15));
    }
```

Project Object Model – Login Page

Project layout for EaseMyTrip.com automation suite

The Login Page class encapsulates user's Email I'D/Mobile Number , Password and submit button locators, exposing simple action methods (enterUsername , enterPassword, clickLogin) and getSuccessMessage() accessor used by the test layer for assertions.

```
package com.Framework.TestCase;

import org.openqa.selenium.support.PageFactory;

Run All
public class LoginEaseMyTrip extends BaseClass
{
    @Test
    Run | Debug
    public void loginApp() throws Exception
    {

        LoginPages loginPage = PageFactory.initElements(driver, LoginPages.class);

        Thread.sleep(2000);

        String username = excel.getStringData("Login", 0, 0);
        String password = excel.getStringData("Login", 0, 1);
        Thread.sleep(5000);
        loginPage.login_SauceLabs(username, password);
        Thread.sleep(5000);
    }
}
```

TestNG Test Case & Excel Data Utility

Project layout for EaseMyTrip.com automation suite

LoginTest (extending BaseClass) pulls test data from an Excel sheet through ExcelUtility, drives the LoginPage flow, and uses TestNG's Assert to verify the success message — keeping test data fully decoupled from script logic.

```java
package utility;

import java.io.File;

public class ExcelDataProvider
{
    XSSFWorkbook wb;

    public ExcelDataProvider()
    {
        try
        {
            File src = new File("C:\\Users\\Nirmalya.Mohanta\\OneDrive - Coforge Limite
            FileInputStream fis = new FileInputStream(src);
            wb = new XSSFWorkbook(fis);
        }
        catch (Exception ex)
        {
            System.out.println("Unable to load Excel file: " + ex.getMessage());
        }
    }

    public String getStringData(String sheetName, int row, int col)
    {
        return wb.getSheet(sheetName).getRow(row).getCell(col).getStringCellValue();
    }

    public double getNumericData(String sheetName, int row, int col)
    {
        return wb.getSheet(sheetName).getRow(row).getCell(col).getNumericCellValue();
    }
}
```

Configuration Reader & Project Management

Project layout for EaseMyTrip.com automation suite

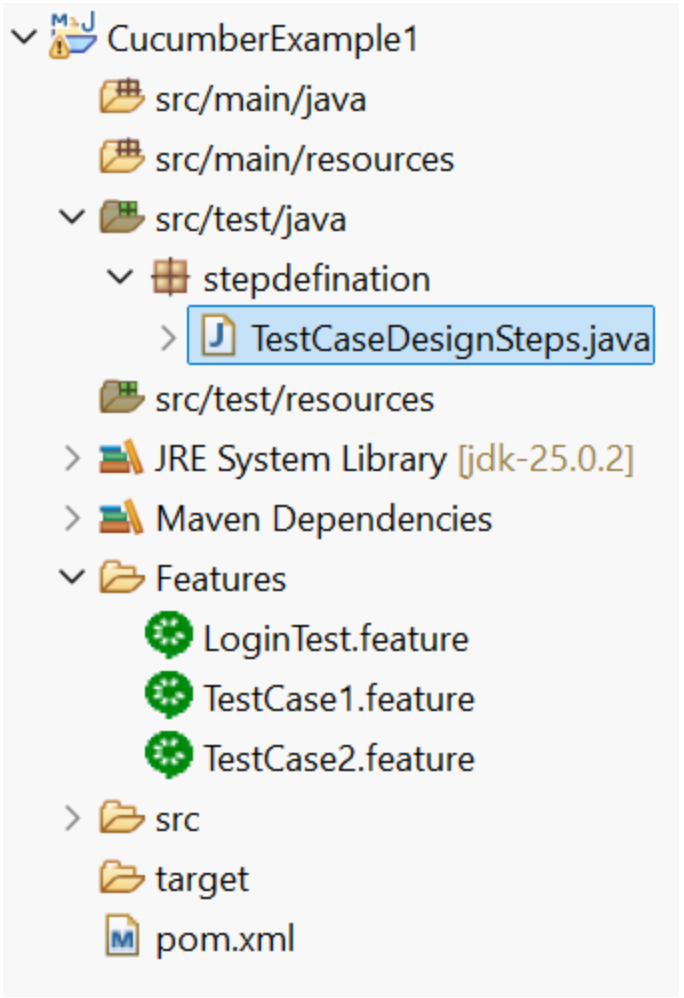
ReadConfig loads environment values (URL, browser, implicit wait) from a properties file, so the framework can be reconfigured for different environments without touching the test code.

```java
package utility;

import java.io.File;

public class ConfigDataProvider
{
    Properties pro;

    public ConfigDataProvider()
    {
        File src = new File("./configuration/config.properties");
        try
        {
            FileInputStream file = new FileInputStream(src);
            pro = new Properties();
            pro.load(file);
        }
        catch (Exception ex)
        {
            System.out.println("Unable to load config file: " + ex.getMessage());
        }
    }

    public String getBrowser()
    {
        return pro.getProperty("Browser");
    }

    public String getAppUrl()
    {
        return pro.getProperty("AppUrl");
    }
}
```

Cucumber

- Used for Behavior Driven Development — writes test cases
- Works on top of Gherkin — a structured, human-readable syntax for writing feature files (.feature).
- Generates readable HTML/JSON reports showing pass/fail per scenario step.
- Works well with TestNG/JUnit runners for execution and CI/CD integration (e.g., Jenkins).
- Supports Scenario Outline with Examples tables for data-driven testing.

Cucumber – Project Structure & Step Defination




```java
package stepdefination;

import org.openqa.selenium.By;

public class TestCaseDesignSteps {
    WebDriver driver;

    // ---------- Test Case 1 (Valid Login)

    @Given("User should open Chrome Browser")
    public void user_should_open_chrome_browser() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @When("User should Enter url in Browser")
    public void user_should_enter_url_in_browser() {
        driver.get("https://practicetestautomation.com/practice-test-login/");
    }

    @When("User should Navigate Home Page")
    public void user_should_navigate_home_page() {
        System.out.println("Navigated to login page");
    }

    @When("Enter UserName and Password in Edit Box")
    public void enter_user_name_and_password_in_edit_box() {
        driver.findElement(By.id("username")).sendKeys("student");
        driver.findElement(By.id("password")).sendKeys("Password123");
    }
```

Cucumber- Gherkin Feature File

Each .feature file describes one piece of application behaviour in plain English, structured as a Feature with one or more Scenarios composed of Given/When/Then/Andsteps — fully readable by non-technical stakeholders.

```
Feature: Checking login Functionality  Run | Debug | Profile

  Scenario: Successful Login with Valid Credentials  Run | Debug | Profile
    Given User should open Chrome Browser
    When User should Enter url in Browser
    When User should Navigate Home Page
    And Enter UserName and Password in Edit Box
    And Click On Login PushButton
    Then Message displayed Login Successfully
```

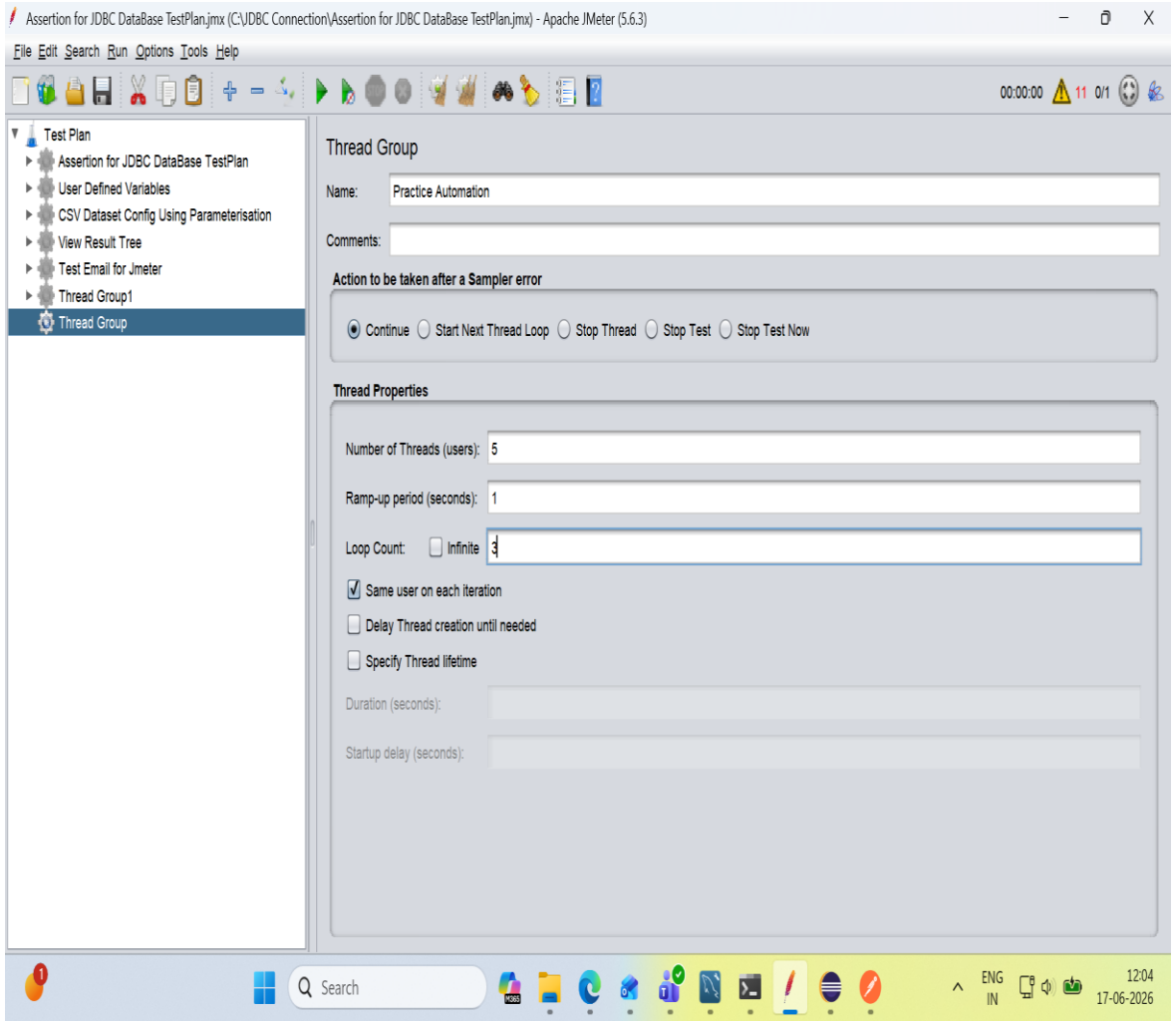
Cucumber – Maven Dependencies

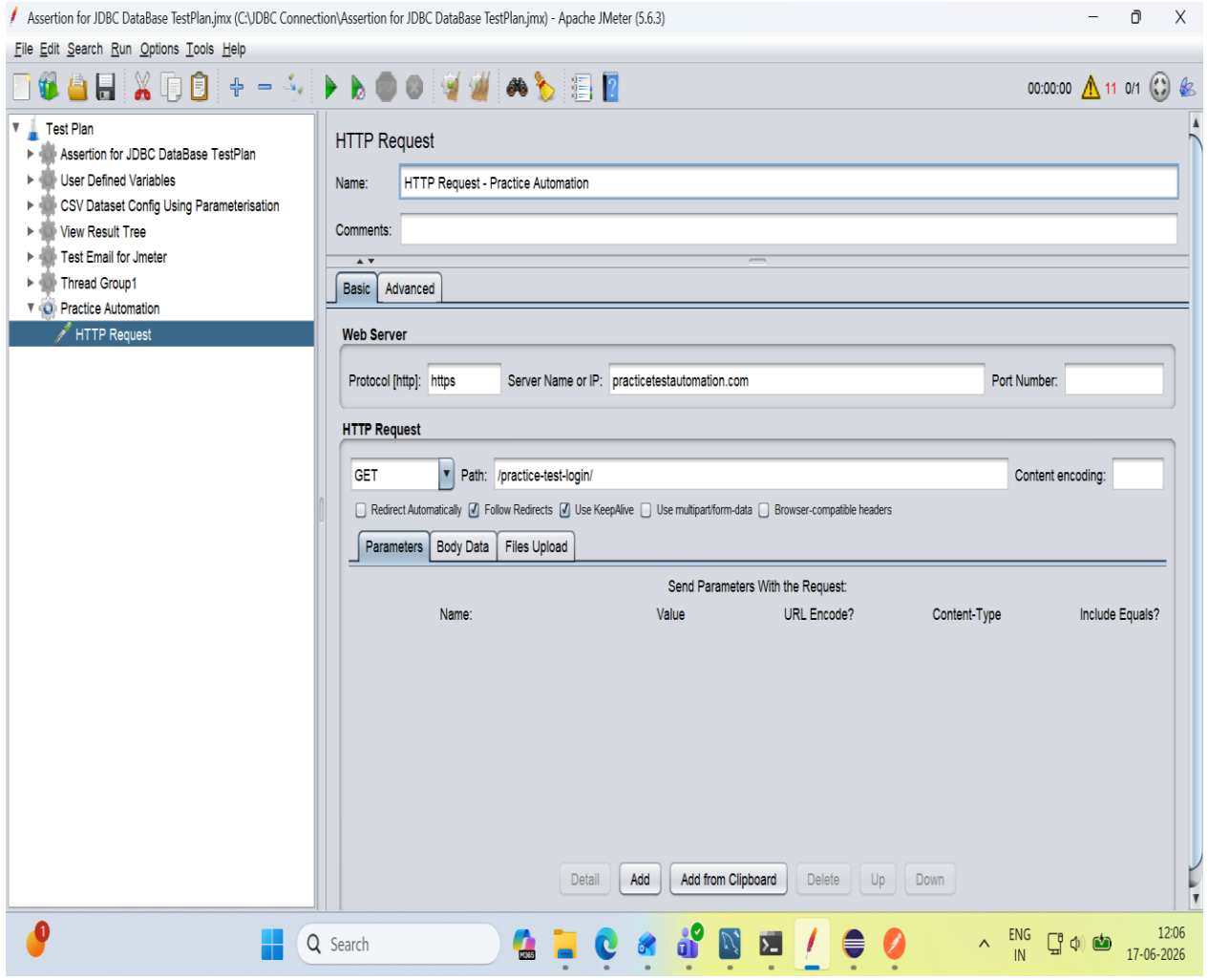
```xml
<project xmlns="https://maven.apache.org/POM/4.0.0"
    xmlns:xsi="https://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>CucumberExample1</groupId>
    <artifactId>CucumberExample1</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
        <dependencies>
            <!-- Source:
        https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
            <dependency>
                <groupId>org.seleniumhq.selenium</groupId>
                <artifactId>selenium-java</artifactId>
                <version>4.44.0</version>
                <scope>compile</scope>
            </dependency>

            <!-- Source:
            https://mvnrepository.com/artifact/io.cucumber/cucumber-core -->
            <dependency>
                <groupId>io.cucumber</groupId>
                <artifactId>cucumber-core</artifactId>
                <version>7.34.3</version>
                <scope>compile</scope>
            </dependency>

            <!-- Source:
            https://mvnrepository.com/artifact/io.cucumber/cucumber-java -->
            <dependency>
                <groupId>io.cucumber</groupId>
                <artifactId>cucumber-junit</artifactId>
                <version>7.34.3</version>
                <scope>test</scope>
            </dependency>

            <!--
            https://mvnrepository.com/artifact/io.cucumber/cucumber-jvm-deps -->
            <dependency>
                <groupId>io.cucumber</groupId>
                <artifactId>cucumber-jvm-deps</artifactId>
                <version>1.0.6</version>
                <scope>provided</scope>
            </dependency>

            <!-- https://mvnrepository.com/artifact/junit/junit -->
            <dependency>
                <groupId>junit</groupId>
                <artifactId>junit</artifactId>
                <version>3.8.1</version>
                <scope>test</scope>
            </dependency>

            <!--
        https://mvnrepository.com/artifact/net.masterthought/maven-cucumber-reporting -->
            <dependency>
                <groupId>net.masterthought</groupId>
                <artifactId>maven-cucumber-reporting</artifactId>
                <version>4.3.0</version>
            </dependency>
        </dependencies>

</project>
```

JMeter

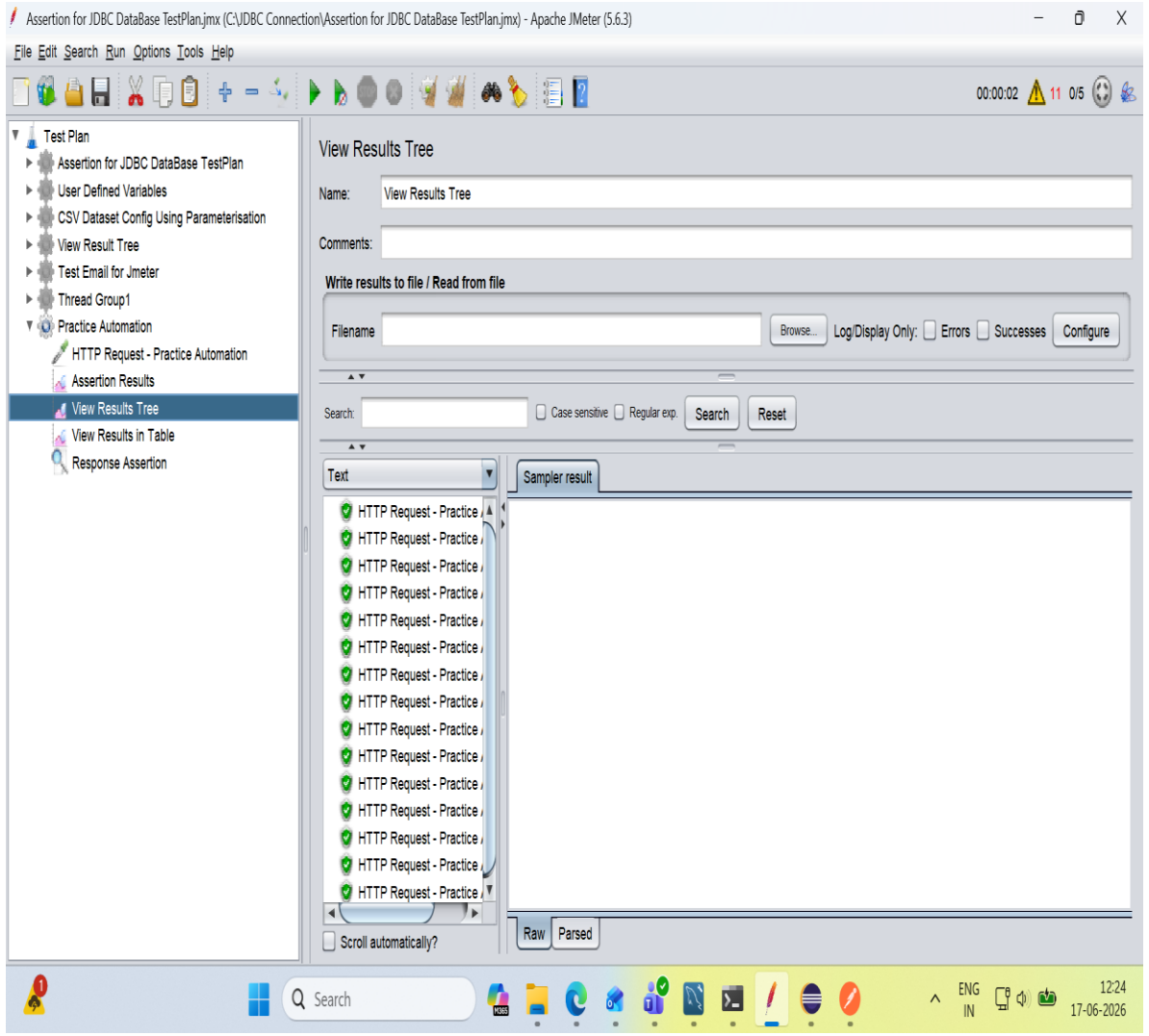
- It's a free, open-source tool built specifically for load and performance testing.
- Simulates hundreds/thousands of virtual users hitting an application simultaneously to see how it holds up under real-world traffic.
- Tracks how fast the server reacts and how much load it can take before slowing down.
- Works across web apps, APIs, and databases — not limited to just websites.
- Lets teams catch bottlenecks and crashes early, before users ever experience them in production.

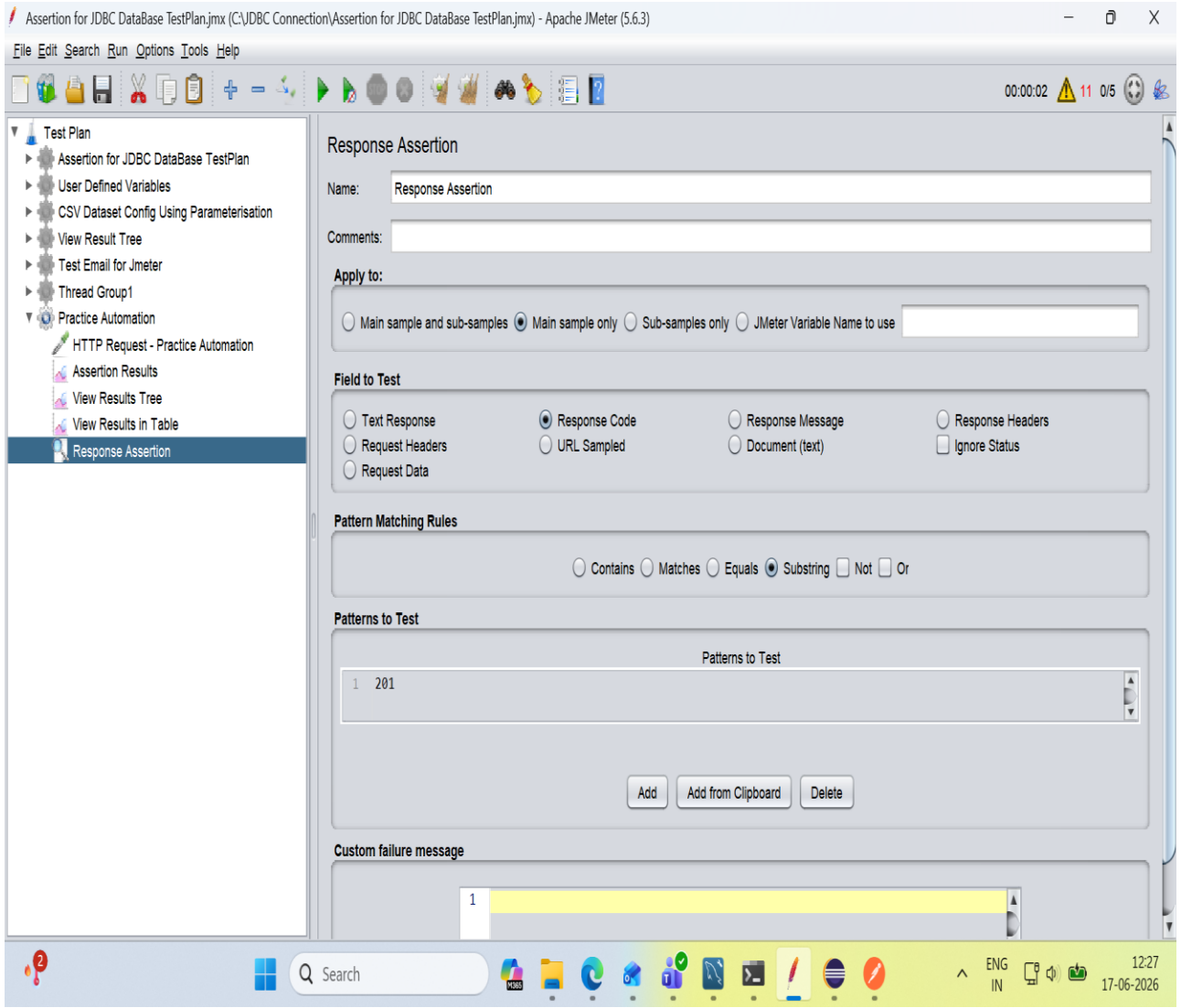
JMeter – Test Plan Configuration



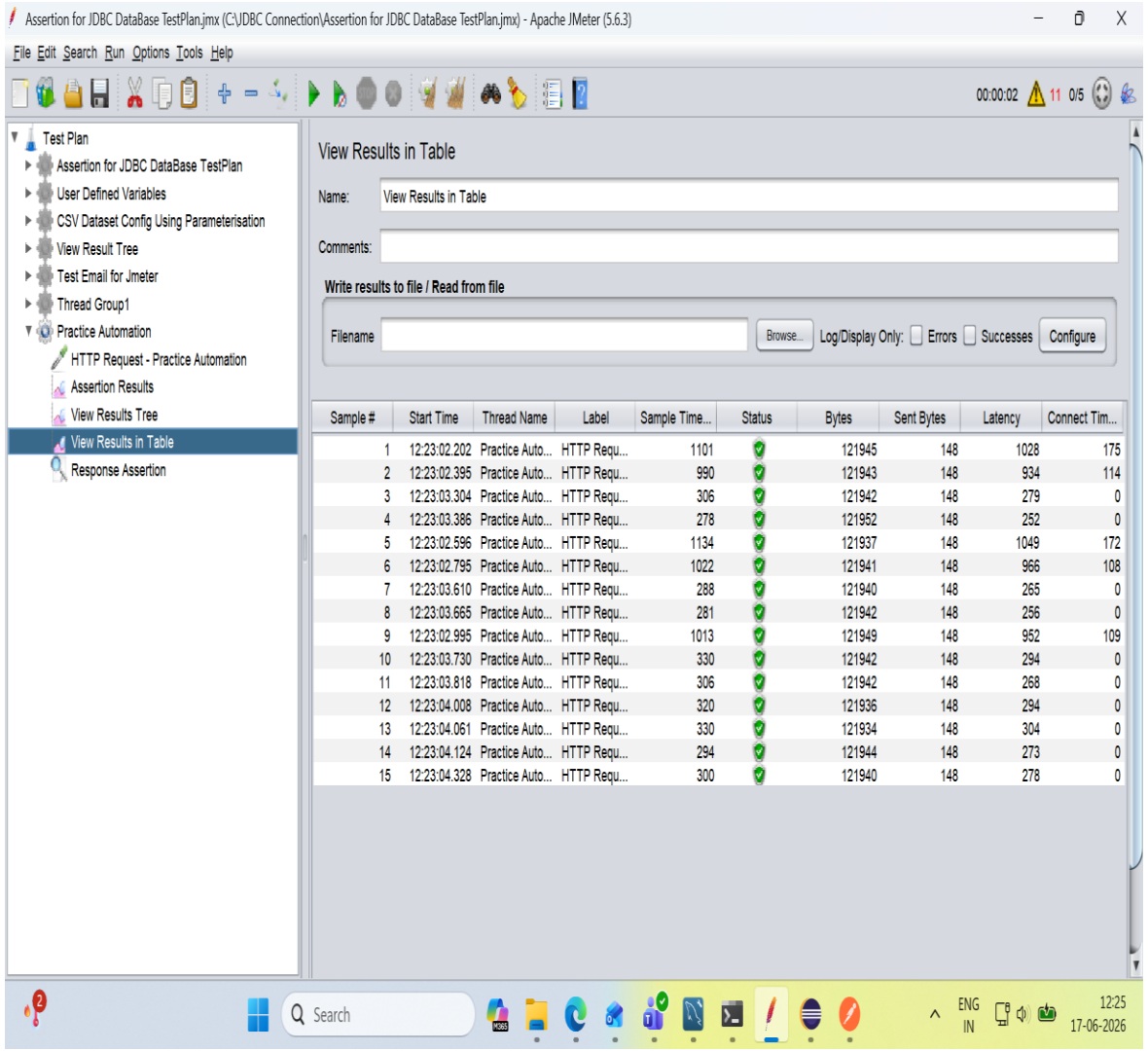


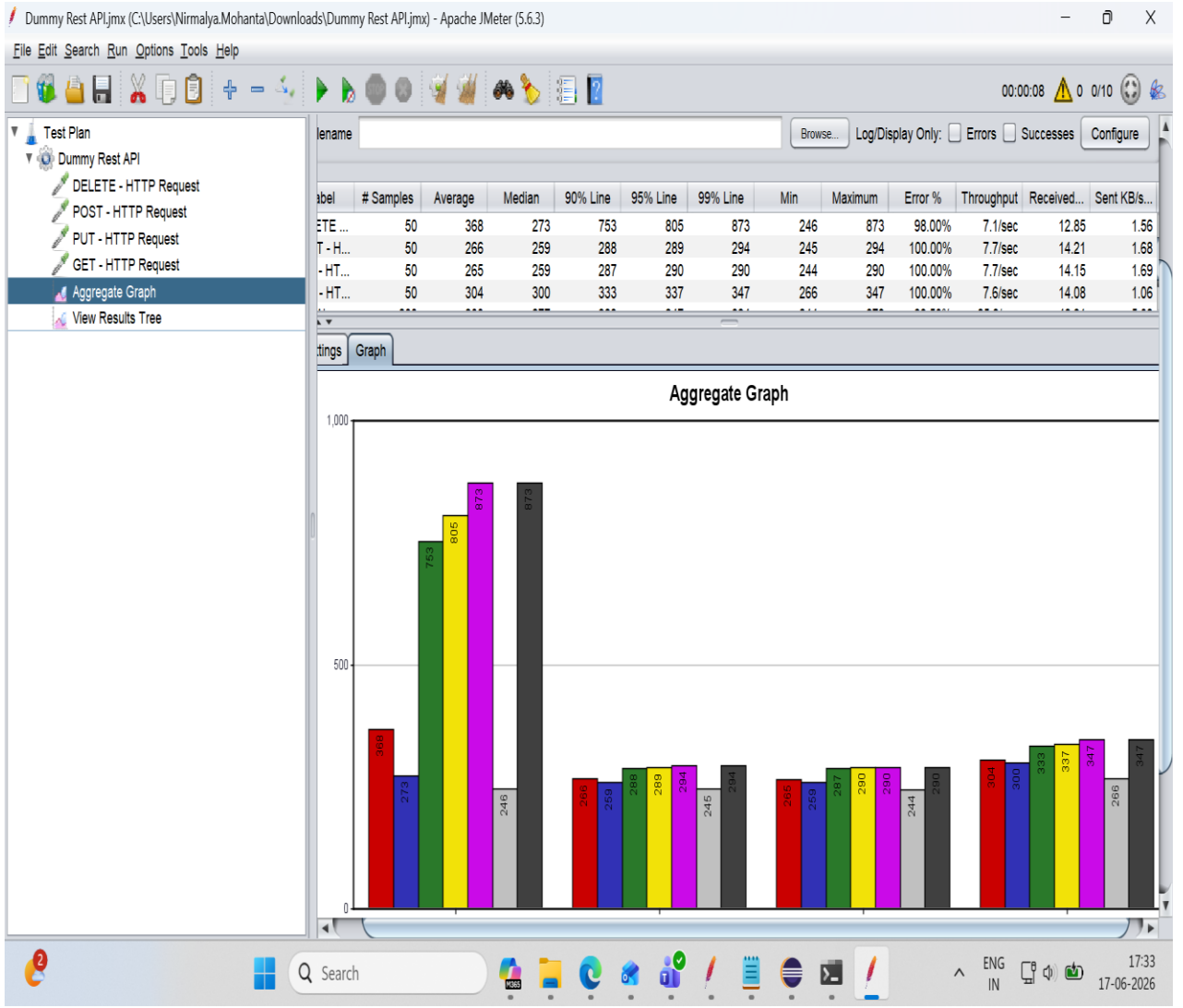
JMeter – Thread Group & HTTP Request Sampler





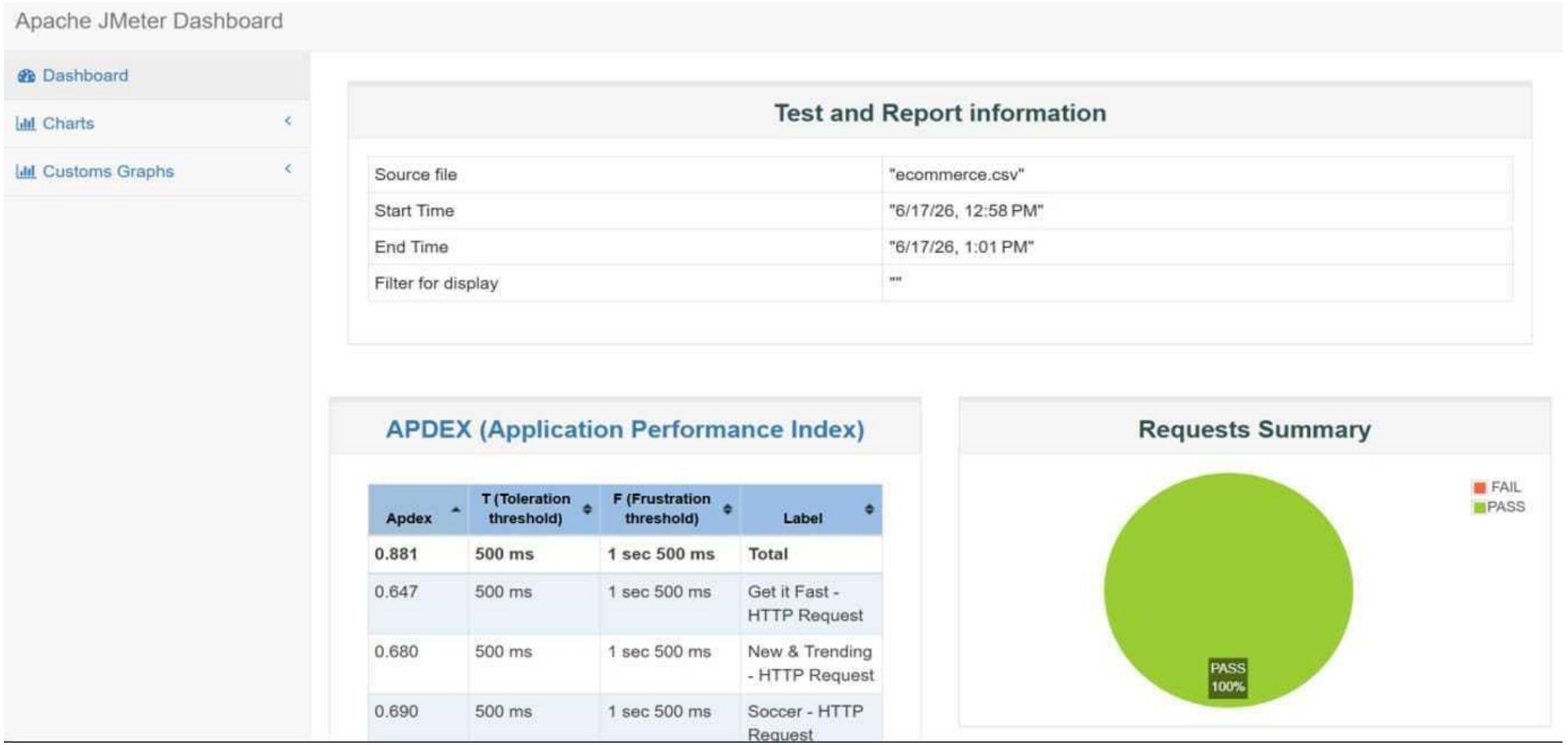
JMeter – Listeners & Result Visualization





JMeter – HTML Dashboard Report

The test plan was executed using JMeter's non-GUI mode, resulting in an HTML performance report that includes APDEX metrics, response time evaluations, and a detailed summary of successful and failed requests.

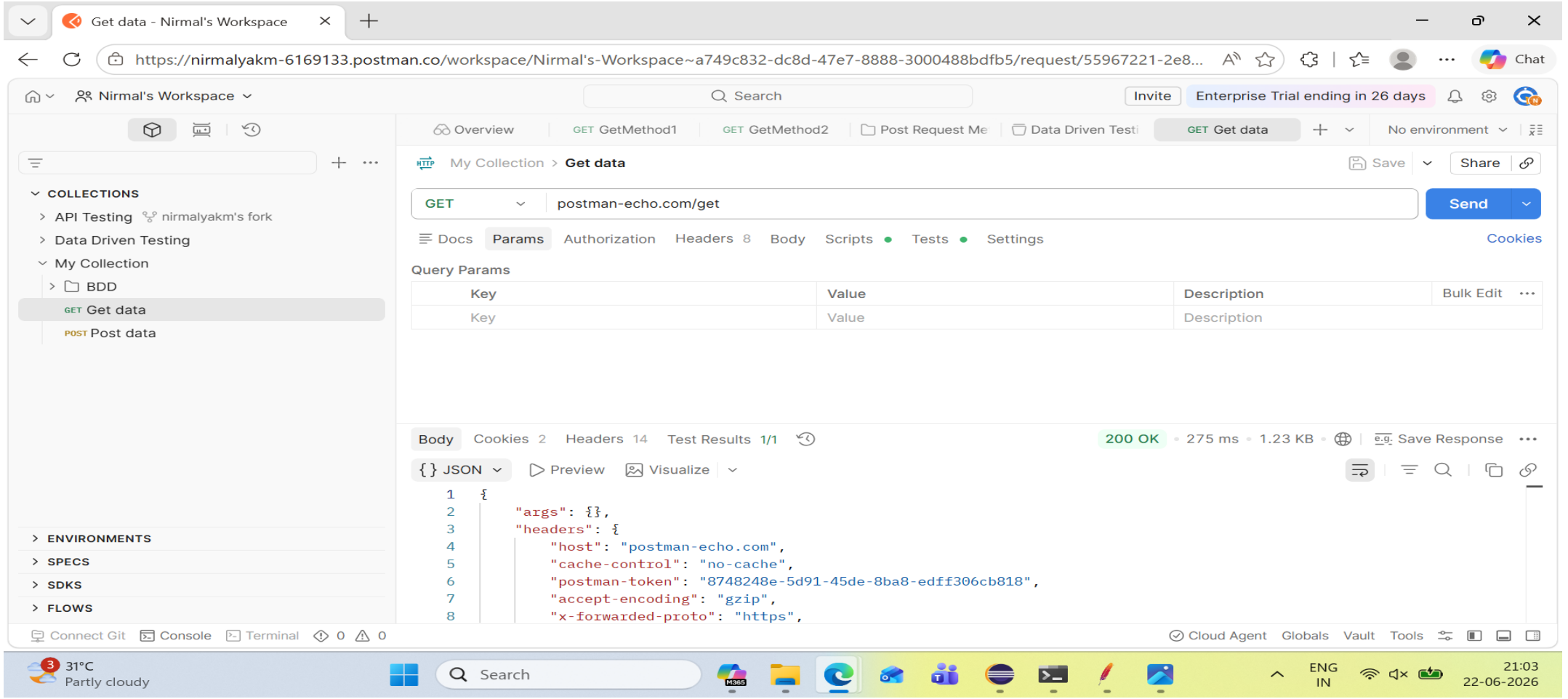


Postman and Rest Assured

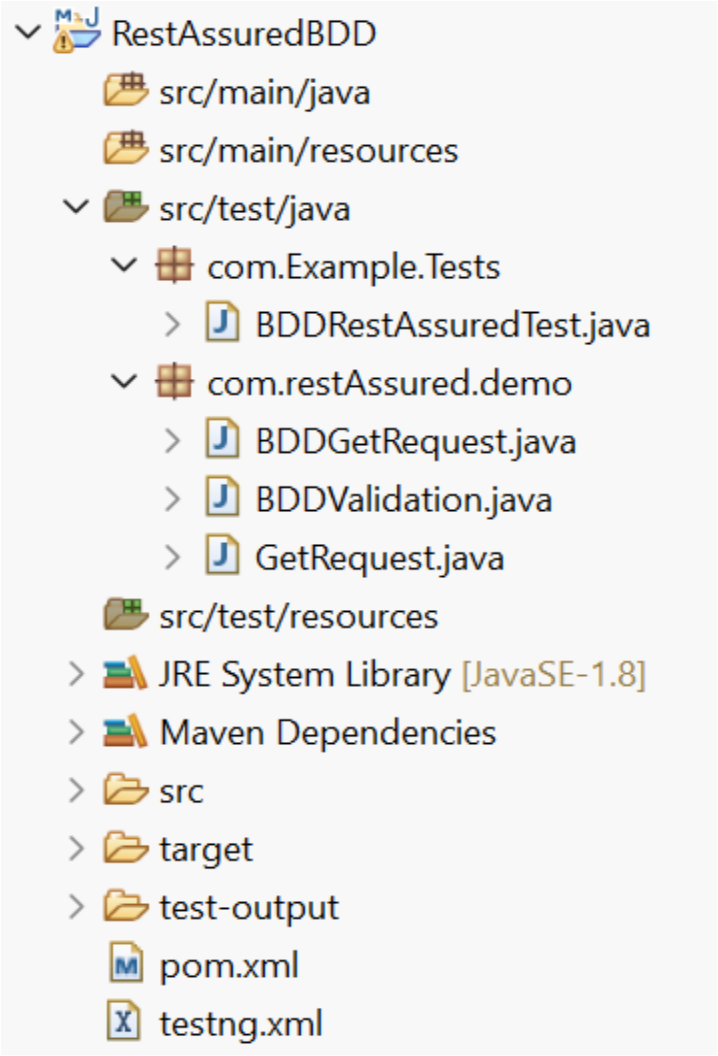
- Postman is a widely used application that enables users to interact with and verify API endpoints manually.
- With Postman, API requests such as GET, POST, PUT, and DELETE can be executed without the need to write programming code.
- REST Assured is a Java-based framework designed specifically for automating API testing tasks.
- Postman is primarily preferred for manual API validation and exploratory testing, while REST Assured is mainly used for automated API testing and regression testing.
- REST Assured integrates seamlessly with automation frameworks such as TestNG and JUnit, making it ideal for continuous testing environments.
- Postman provides a user-friendly graphical interface, making it suitable for beginners and quick API checks.
- Together, Postman and REST Assured help ensure API reliability by supporting both manual verification and automated validation processes.

Postman – Manual API Execution

A POST request is executed using Postman to test the API endpoint. The response body, status code, headers, and response time are reviewed to verify that the API is functioning correctly and returning the expected results.



Rest Assured – Automation Framework



- The RestAssuredWithPriority project is developed as a Maven-based framework, ensuring a standardized and maintainable approach to API test automation.
- CompletePutPostTest and DataDrivenTesting are designed to validate API functionality by covering various request and response scenarios.
- RestAssuredPriority manages the sequence of test execution, ensuring API test cases run in the intended order across the test suite.
- The framework promotes organized test management, efficient execution, and improved reusability of API test scripts.

Rest Assured – Sample Test Script

```java
package com.Example.Tests;

import org.json.JSONObject;

Run All
public class BDDRestAssuredTest {
    @BeforeClass
    public void setup() {
        RestAssured.baseURI = "https://jsonplaceholder.typicode.com";
    }

    @Test
    Run | Debug
    public void testPostRequestBDD() {
        System.out.println("=== BDD APPROACH ===\n");

//      1. Create JSON Object
        JSONObject requestBody = new JSONObject();
        requestBody.put("title","Test Post BDD");
        requestBody.put("body","This is a test post body using BDD");
        requestBody.put("userId",2);

        System.out.println("Request Body: " + requestBody.toString());

//      BDD Style Test - Given, When, Then
        given()
        .log().all()  // Log Request Details
        .header("Content-Type", "application/json")
        .body(requestBody.toString())
        .when()
        .post("/posts")
        .then().log().all()
        .statusCode(201)
        .contentType("application/json")
        .body("title", equalTo("Test Post BDD"))
        .body("body", equalTo("This is a test post body using BDD"))
        .body("userId", equalTo(2)).body("id", notNullValue())
        .time(lessThan(8000L));

        System.out.println("\n== BDD APPROACH COMPLETED ===");


    }
}
```

GitHub Repository Links

Module Name:- GitHub Repositories			Submitted By:- Nirmalya Kumar Mohanta
No of Repositories :- 5			Submitted To:- Raghavendra BN
Emp ID:- 85009868			
S.NO	**Date**	**Project Name**	**Repository Link**
1	01/06/2026	DataDrivenTestingUsingApachePOI	https://github.com/Nirmalyakumar/BootCampTesting-HYD/tree/main/DataDrivenTesting-UsingApachePOI
2	01/06/2026	DataDrivenTestingUsingCSV	https://github.com/Nirmalyakumar/BootCampTesting-HYD/blob/main/DataDrivenTestingUsingCSVFile/src/com/DataDrivenTesting/CSVDataDrivenTesting.java
3	02/06/2026	DataProvideInTestNG	https://github.com/Nirmalyakumar/BootCampTesting-HYD/tree/main/DataProviderinTestNG
4	02/06/2026	DifferentBrowserTestCaseStudy	https://github.com/Nirmalyakumar/BootCampTesting-HYD/tree/main/Launch%20Different%20Browsers/src/com
5	02/06/2026	LaunchDifferentWebsites	https://github.com/Nirmalyakumar/BootCampTesting-HYD/tree/main/LaunchDifferentWebsites-Examples
6	03/06/2026	HybridDrivenFrameWork	https://github.com/Nirmalyakumar/BootCampTesting-HYD/tree/main/HybridDrivenFramework
7	04/06/2026	ParameterTestNG	https://github.com/Nirmalyakumar/BootCampTesting-HYD/tree/main/Assignments/src/com/loginPage
8	04/06/2026	PageObjectModelEasyCalculation	https://github.com/Nirmalyakumar/BootCampTesting-HYD/tree/main/PageObjectModelEasyCalculation

Thank You!

Manual Testing: Lenskart.com | Automation Testing: EaseMyTrip.com

Nirmalya Kumar Mohanta | Graduate Engineer Trainee, Coforge | Emp ID: 85009868